

MécaDéetect - Application de suivi de maintenance industrielle

25 septembre 2026 · Mathieu FENOUIL

"Le carnet de santé des machines" - Permettre à un site industriel de déclarer les pannes, planifier les interventions de maintenance et piloter l'état de son parc d'équipements.

Vision produit

Un opérateur de production constate qu'une presse hydraulique est à l'arrêt. Il ouvre MécaDéetect, sélectionne l'équipement et déclare la panne avec une priorité. L'équipement passe automatiquement « en panne ». Le responsable maintenance voit la demande sur son tableau de bord, planifie l'intervention et l'assigne à un technicien habilité sur ce type de machine. Le technicien démarre l'intervention depuis sa tablette, puis la clôture avec un compte rendu et le temps passé. L'équipement repasse « en service ». Chaque changement de statut est tracé, et le responsable suit en temps réel les indicateurs du parc : pannes par équipement, temps moyen de résolution, charge des techniciens.

Acteurs

Acteur	Rôle
Demandeur	Opérateur de production : déclare une panne sur un équipement, suit ses demandes
Technicien	Consulte ses interventions assignées, les démarre, les clôture avec un compte rendu
Responsable maintenance (Admin)	Gère les équipements et les techniciens, planifie, assigne et annule les interventions, consulte le tableau de bord et exporte les données

Stack technique imposée

Frontend

Technologie	Usage
Angular	Framework front, composants standalone, routing avec lazy loading
TypeScript	Typage statique, interfaces alignées sur les DTO de l'API
Signals	Gestion de l'état local des composants
RxJS	Appels HTTP, flux asynchrones
Reactive Forms	Formulaires avec validation
Angular Material	Composants d'interface, design responsive

Backend

Technologie	Usage
Java 25	Dernière version LTS, langage principal : records, sealed interfaces, pattern matching, virtual threads

Technologie	Usage
Spring Boot	Architecture Controller → Service → Repository, DTOs
Spring Data JPA / Hibernate	Accès aux données, requêtes JPQL et natives, pagination
Spring Security	Sécurisation des API, gestion des rôles
JWT	Authentification stateless
Bean Validation	Validation des données entrantes
Maven	Build et gestion des dépendances

Base de données

Technologie	Usage
Oracle Database Free	Base de données relationnelle
PL/SQL	Trigger d'historisation, package métier, génération de données
Flyway ou Liquibase	Migrations versionnées du schéma
Looping	Modélisation MCD / MLD (méthode Merise)

DevOps

Technologie	Usage
Docker	Conteneurisation de la base, de l'API et du front
Docker Compose	Environnement complet lancé en une commande, en local comme en production
Nginx	Service du front Angular
Caddy	Reverse proxy, HTTPS automatique avec Let's Encrypt
GitHub Actions	Pipeline build, tests, analyse, images, déploiement
GitHub Container Registry	Stockage des images Docker (arm64)
Oracle Cloud (OCI) Always Free	Hébergement : VM Ampere A1 (2 OCPU, 12 Go RAM)

Tests

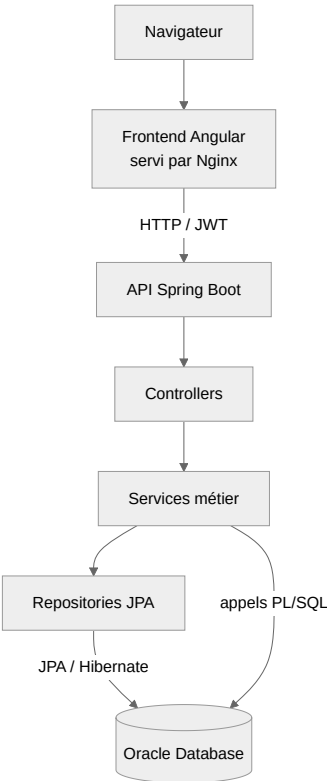
Technologie	Usage
JUnit 5	Tests unitaires backend, méthodologie TDD sur les règles métier
AssertJ	Assertions fluides et lisibles, en complément de JUnit
Mockito	Mocks et tests de services
Testcontainers	Tests d'intégration sur une vraie base Oracle
MockMvc	Tests des endpoints et de la sécurité
Gatling	Tests de performance et de charge, rapports avant/après optimisation
Outils de test Angular	Tests des composants et services front
Robot Framework (optionnel)	Tests end-to-end des parcours utilisateur
Postman	Tests API manuels et collections

Outils

Outil	Usage
OpenAPI 3 (contract-first)	Contrat d'API <code>openapi.yaml</code> écrit avant le code, source de vérité de l'API
OpenAPI Generator	Génération des interfaces Spring et du client Angular typé à partir du contrat
Swagger UI	Consultation interactive du contrat OpenAPI
SonarCloud / JaCoCo	Qualité et couverture du code
OWASP Dependency-Check / Dependency-Track	Détection des vulnérabilités dans les dépendances
Trivy	Analyse de sécurité des images Docker
Claude Code	Assistance IA au développement
Visual Studio Code / IntelliJ IDEA	IDE
GitHub	Gestion du dépôt, pull requests

Architecture monolithique en couches

Contrairement à JuriBook, MécaDétekt est un monolithe Spring Boot : le périmètre fonctionnel ne justifie pas de microservices, et ce choix montre la capacité à adapter l'architecture au besoin.



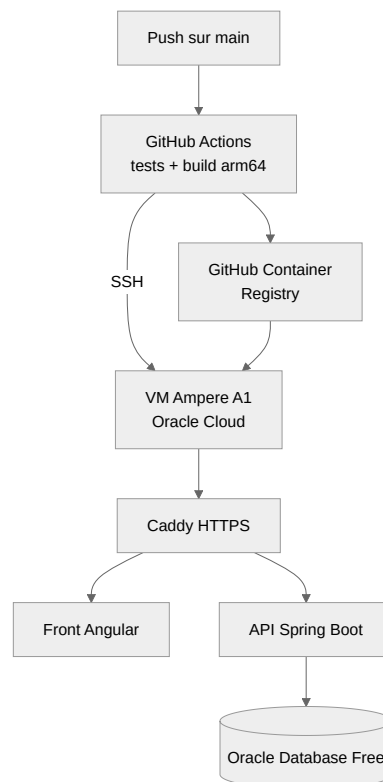
Composant	Responsabilité
frontend	Interface Angular : écrans par rôle, appels à l'API, gestion du JWT
controller	Endpoints REST, validation des entrées, DTO en records Java

Composant	Responsabilité
service	Règles métier, cycle de vie des interventions, transactions
repository	Accès aux données avec Spring Data JPA, requêtes JPQL et natives
domain	Entités JPA et types métier (statuts, priorités)
security	Configuration Spring Security, filtre JWT, rôles
Oracle	Stockage, contraintes d'intégrité, trigger et package PL/SQL

Le code est organisé par domaine métier (package-by-feature : `auth`, `equipement`, `intervention`, `dashboard`), chaque domaine reprenant ces couches. Les erreurs sont centralisées dans un `@RestControllerAdvice` au format Problem Details (RFC 9457).

Déploiement sur Oracle Cloud

MécaDéetect est déployé gratuitement sur l'offre Always Free d'Oracle Cloud Infrastructure : une base Oracle hébergée sur le cloud d'Oracle, avec le même Docker Compose qu'en local.



Élément	Choix
Hébergeur	Oracle Cloud Infrastructure, offre Always Free
Serveur	1 VM Ampere A1 (ARM64), 2 OCPU, 12 Go de RAM
Région	Marseille, ou Paris si la capacité Ampere A1 est indisponible
Conteneurs	Oracle Database Free, API Spring Boot, front Angular (Nginx), Caddy
HTTPS	Caddy avec certificat Let's Encrypt automatique
Livraison	GitHub Actions : tests, build des images arm64, publication sur GHCR, mise à jour de la VM par SSH
Secrets	Variables d'environnement sur la VM et secrets GitHub, jamais dans le dépôt

Points de vigilance :

- **Architecture ARM64** : toutes les images doivent exister en arm64. L'image Oracle Database Free en version ARM est à vérifier au Sprint 0.
- **Région d'origine** : les ressources Always Free ne se créent que dans la région choisie à l'inscription, qui ne peut plus être modifiée.
- **Plan B pour la base** : si l'image ARM pose problème, utiliser une instance Oracle Autonomous Database, incluse dans l'offre gratuite.
- **Sauvegarde** : export régulier de la base (Data Pump) vers le stockage objet d'OCI.

Cas d'usage base de données - Justification détaillée

Oracle n'est pas utilisé comme simple stockage. Chaque cas d'usage ci-dessous exploite une technique précise et répond à un problème réel d'une application de maintenance, sur une base chargée avec 5 ans d'historique (500 000 interventions).

UC-DB1 — Historique immuable des statuts (Trigger PL/SQL)

Problème : Un technicien affirme avoir démarré une intervention à 8 h, le responsable soutient qu'elle n'a commencé qu'à 11 h. Qui dit vrai ?

Solution : Un trigger `TRG_INTERVENTION_HISTO` s'exécute après chaque changement de statut et insère une ligne dans `HISTORIQUE_STATUT` (ancien statut, nouveau statut, auteur, horodatage). L'auteur est lu dans la colonne `id_modifie_par` de l'intervention, que l'application renseigne avec l'utilisateur connecté à chaque changement de statut. L'historique est ainsi écrit par la base elle-même, quel que soit le chemin de modification.

Technique : Trigger AFTER UPDATE, table en ajout seul.

Valeur métier : Traçabilité complète des interventions, preuve en cas de litige.

UC-DB2 — Tableau de bord et reporting (Fonctions analytiques)

Problème : Le responsable veut savoir quels équipements tombent le plus souvent en panne et si l'activité augmente d'un mois sur l'autre.

Solution : Des requêtes SQL de reporting calculent le top 10 des équipements en panne avec `RANK()`, l'évolution mensuelle avec `LAG()`, le temps moyen de résolution par priorité et la charge par technicien. Les résultats sont projetés dans des DTO, jamais chargés comme entités complètes.

Technique : Jointures, agrégations, fonctions analytiques, projections DTO.

Valeur métier : Pilotage du parc et priorisation du remplacement des machines.

UC-DB3 — Performance sur forte volumétrie (Index et plans d'exécution)

Problème : Avec 500 000 interventions, l'écran « Interventions ouvertes » et l'historique d'un équipement deviennent lents.

Solution : Chaque requête critique est analysée avec `EXPLAIN PLAN / DBMS_XPLAN`. Des index ciblés sont ajoutés, dont un index sur chaque clé étrangère (Oracle ne les crée pas automatiquement). Temps et plans avant/après sont consignés dans `docs/PERFORMANCE.md`.

Technique : Index composites, lecture des plans d'exécution, mesure.

Valeur métier : Application réactive même après des années d'utilisation.

UC-DB4 — Archivage des anciennes interventions (Package PL/SQL)

Problème : La table des interventions grossit sans cesse, alors que les interventions de plus de 3 ans ne sont presque jamais consultées.

Solution : La procédure `PKG_MAINTENANCE.archiver_interventions` déplace les interventions terminées ou annulées avant une date vers une table d'archive, par lots (`BULK COLLECT / FORALL`), avec gestion des exceptions et journal d'exécution. Elle est déclenchée depuis l'API par un administrateur.

Technique : Package PL/SQL, traitement en masse, appel depuis Spring (`SimpleJdbcCall`).

Valeur métier : Base allégée et performances stables dans le temps.

UC-DB5 — Liste des interventions sans surcharge (Problème N+1)

Problème : Afficher 20 interventions avec leur équipement et leur technicien génère 41 requêtes SQL au lieu d'une.

Solution : Le problème est mis en évidence dans les logs SQL d'Hibernate, puis corrigé avec `JOIN FETCH` ou `@EntityGraph`, combiné à la pagination. Le nombre de requêtes avant/après est documenté.

Technique : Chargement LAZY, fetch join, pagination, logs Hibernate.

Valeur métier : Écrans de liste rapides, charge réduite sur la base.

UC-DB6 — Modifications concurrentes (Verrouillage optimiste)

Problème : Deux responsables assignent la même intervention à deux techniciens différents au même moment.

Solution : Une colonne `version` gérée par `@Version` détecte la modification concurrente : la seconde mise à jour est rejetée avec une erreur 409, et l'utilisateur est invité à recharger.

Technique : Verrouillage optimiste JPA.

Valeur métier : Aucune assignation écrasée silencieusement.

Récapitulatif des cas d'usage

Cas d'usage	Technique	Où
UC-DB1 Historique des statuts	Trigger AFTER UPDATE	Oracle (PL/SQL)
UC-DB2 Reporting	RANK(), LAG(), agrégations, projections DTO	Repository (requêtes natives)
UC-DB3 Performance	Index, EXPLAIN PLAN	Migrations + PERFORMANCE.md
UC-DB4 Archivage	Package, BULK COLLECT / FORALL	Oracle (PL/SQL) + appel Spring
UC-DB5 N+1	JOIN FETCH / @EntityGraph	Repository JPA
UC-DB6 Concurrency	@Version	Entité JPA

Stratégie de tests et de qualité

Les tests sont pensés dès la conception : les règles métier sont développées en TDD, et la performance comme la sécurité sont vérifiées automatiquement.

Niveau	Outils	Ce qui est vérifié
Règles métier (TDD)	JUnit 5, AssertJ	Chaque transition du cycle de vie et chaque règle RG-01 à RG-09, test écrit avant le code

Niveau	Outils	Ce qui est vérifié
Services	JUnit 5, Mockito, AssertJ	Logique métier isolée de la base
Intégration	Testcontainers (Oracle)	Repositories, requêtes de reporting, trigger et package PL/SQL
API	MockMvc	Respect du contrat OpenAPI, codes de retour, sécurité par rôle
Performance	Gatling	Temps de réponse sous charge, avant et après indexation (UC-DB3)
Sécurité	OWASP Dependency-Check, Trivy	Vulnérabilités des dépendances et des images, bloquant en CI si critique
Front-end	Outils de test Angular	Formulaires, badges, boutons d'action selon le rôle et le statut
Bout en bout (optionnel)	Robot Framework	Parcours complet : déclaration, planification, démarrage, clôture

Démarche TDD : pour chaque règle de gestion, un test rouge décrit le comportement attendu, puis le code minimal le fait passer, puis le code est remanié. Les règles déjà rédigées dans ce document servent directement de base aux tests.

Modèle de données et règles métier

Modèle de données

Le modèle compte 7 tables, issues de 6 entités et 9 associations, dont une association N:N porteuse (habilitations des techniciens) et une table d'historique alimentée par trigger. Le MCD et le MLD sont réalisés dans Looping ; le fichier source et les images exportées sont versionnés dans `docs/modelisation/`. UTILISATEUR est relié quatre fois à d'autres entités, avec un rôle distinct à chaque fois : demandeur, technicien, auteur de la modification et auteur de l'historique.

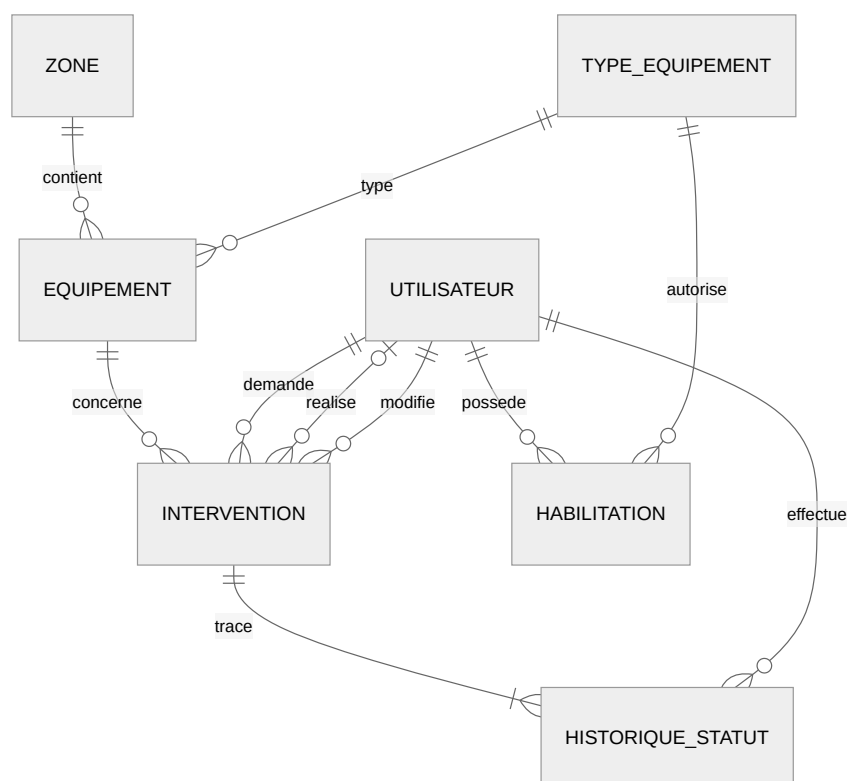
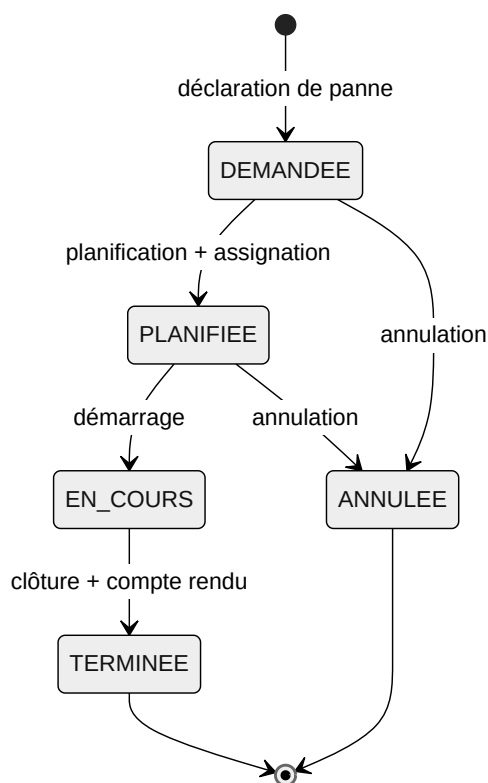


Table	Colonnes (# = clé étrangère)	Contraintes
ZONE	id_zone, code, libelle	code unique
TYPE_EQUIPEMENT	id_type, libelle	libelle unique
EQUIPEMENT	id_equipement, reference, nom, date_mise_service, statut, #id_type, #id_zone	reference unique, CHECK statut, 2 FK
UTILISATEUR	id_utilisateur, nom, prenom, email, mot_de_passe_hash, role, actif	email unique, CHECK role
HABILITATION	#id_type, #id_technicien, date_habilitation	PK composite, 2 FK
INTERVENTION	id_intervention, numero, titre, description, priorite, statut, date_demande, date_planifiee, date_debut, date_fin, temps_passe_min, compte_rendu, motif_annulation, version, #id_equipement, #id_demandeur, #id_technicien, #id_modifie_par	numero unique, CHECK priorite et statut, 4 FK
HISTORIQUE_STATUT	id_historique, ancien_statut, nouveau_statut, date_changement, #id_intervention, #id_auteur	2 FK

Clés primaires par identité Oracle, statuts et priorités en `VARCHAR2` avec contrainte `CHECK`, numéro d'intervention lisible (INT-2026-000123) généré par séquence.

Cycle de vie d'une intervention



Toute transition non prévue est refusée par l'API (409 Conflict). Les transitions sont modélisées en Java 25 avec une sealed interface et du pattern matching.

Règles de gestion

Règle	Description
RG-01	Seul un Admin peut planifier, assigner ou annuler une intervention.
RG-02	Seul le technicien assigné peut démarrer et clôturer son intervention.
RG-03	Une annulation exige un motif ; une clôture exige un compte rendu et un temps passé supérieur à 0.
RG-04	Un technicien ne peut avoir qu'une seule intervention EN_COURS à la fois.
RG-05	Une intervention EN_COURS passe l'équipement EN_MAINTENANCE ; à la clôture ou à l'annulation, il repasse EN_SERVICE s'il n'a plus d'intervention ouverte.
RG-06	Une déclaration de panne passe l'équipement EN_PANNE.
RG-07	Un équipement archivé ne peut plus recevoir de nouvelle intervention.
RG-08	Chaque changement de statut est historisé (UC-DB1).
RG-09	Un technicien ne peut être assigné qu'à un type d'équipement pour lequel il est habilité.

Priorités : BASSE, NORMALE, HAUTE, CRITIQUE. **Statuts d'un équipement :** EN_SERVICE, EN_PANNE, EN_MAINTENANCE, ARCHIVE.

Sprints

Sprint 0 — Setup, modélisation & fondations (1 semaine)

#	Tâche	Critère d'acceptation
0.1	Réaliser le MCD et le MLD dans Looping	Schémas exportés en image, cardinalités validées, ajoutés au README
0.2	Initialiser le dépôt Git (dossiers backend et frontend)	Dépôt créé, README, branches main / develop
0.3	Créer le squelette Spring Boot (Java 25, Maven)	L'API démarre sans erreur, <code>/actuator/health</code> répond UP
0.4	Initialiser le projet Angular avec Angular Material	<code>ng serve</code> affiche une page d'accueil vide
0.5	Configurer Docker Compose avec Oracle Database Free	<code>docker compose up</code> lance Oracle, l'API s'y connecte
0.6	Écrire les migrations initiales du schéma (7 tables, contraintes)	Le schéma est créé automatiquement au démarrage
0.7	Rédiger le contrat <code>openapi.yaml</code> et configurer OpenAPI Generator	Interfaces Spring et client Angular générés au build, Swagger UI affiche le contrat
0.8	Configurer le pipeline GitHub Actions (build + tests)	Un push sur develop déclenche build et tests back et front

Sprint 1 — Authentification & utilisateurs (1 semaine)

#	Tâche	Critère d'acceptation
1.1	Modéliser l'entité Utilisateur et le rôle (DEMANDEUR, TECHNICIEN, ADMIN) en JPA	Entités mappées sur le schéma, enum stocké en texte
1.2	Implémenter le login avec génération JWT	POST /api/v1/auth/login retourne un JWT valide, mot de passe vérifié avec BCrypt
1.3	Protéger les routes par rôle avec Spring Security	Un demandeur reçoit 403 sur une route Admin
1.4	Créer les comptes de démonstration au démarrage	Un compte par rôle disponible en environnement de démo
1.5	Page Angular de connexion et redirection selon le rôle	Formulaire fonctionnel, token stocké, redirection après login
1.6	Intercepteur HTTP et guards de route Angular	Le JWT est ajouté à chaque requête, un 401 renvoie vers /login
1.7	Tests unitaires et d'API de l'authentification	Cas couverts : login valide, mot de passe faux, token invalide, rôle interdit

Sprint 2 — Référentiel équipements (1 semaine)

#	Tâche	Critère d'acceptation
2.1	Modéliser les entités Zone, TypeEquipement, Equipement	Relations JPA correctes, chargement LAZY par défaut
2.2	CRUD équipement (création, modification, archivage logique)	Endpoints réservés à l'Admin, 201 / 200 / 204 selon le cas
2.3	Endpoint de recherche paginée avec filtres (statut, type, zone, texte)	GET /api/v1/equipements?statut=EN_PANNE&zone=A1 retourne une page de résultats
2.4	Gestion centralisée des erreurs (Problem Details)	400, 404 et 409 renvoient un format d'erreur unique
2.5	Page Angular liste des équipements : tableau paginé, filtres, recherche	Filtres et pagination fonctionnels, badges de statut
2.6	Page Angular fiche équipement et formulaire de création / modification	Reactive form avec validation, messages d'erreur de l'API affichés
2.7	Tests unitaires et d'intégration (Testcontainers Oracle)	CRUD, filtres, pagination et contraintes d'unicité testés sur Oracle

Sprint 3 — Interventions & cycle de vie (1 semaine)

#	Tâche	Critère d'acceptation
3.1	Modéliser les entités Intervention, Habilitation et HistoriqueStatut	Relations JPA correctes, colonne <code>version</code> avec <code>@Version</code>
3.2	Modéliser le cycle de vie avec une sealed interface et du pattern matching	Toute transition non prévue lève une exception métier
3.3	API de déclaration de panne	POST /api/v1/interventions retourne 201, statut DEMANDEE, équipement EN_PANNE (RG-06)
3.4	API de planification et d'assignation	POST /api/v1/interventions/{id}/planifier ; technicien non habilité → 409 (RG-09)

#	Tâche	Critère d'acceptation
3.5	API de démarrage et de clôture par le technicien	Seul le technicien assigné peut agir (RG-02), une seule intervention EN_COURS (RG-04)
3.6	API d'annulation avec motif	Annulation impossible sur une intervention TERMINEE, motif obligatoire (RG-03)
3.7	Mise à jour automatique du statut de l'équipement	Les statuts suivent RG-05 à chaque transition
3.8	Gestion de la concurrence (UC-DB6)	Deux assignations simultanées : la seconde reçoit 409
3.9	Pages Angular : liste des interventions, détail, déclaration de panne	Boutons d'action affichés selon le rôle et le statut
3.10	Page Angular « Mes interventions » pour le technicien	Démarrage et clôture en 2 clics, utilisable sur tablette
3.11	Tests du cycle de vie écrits en TDD (JUnit 5 + AssertJ)	Chaque transition autorisée et interdite testée, RG-01 à RG-09 couvertes, tests écrits avant le code

Sprint 4 — Base de données avancée (1 semaine)

#	Tâche	Critère d'acceptation
4.1	Procédure PL/SQL de génération de données (DBMS_RANDOM , FORALL)	500 000 interventions sur 5 ans générées en une commande
4.2	Trigger d'historisation des statuts (UC-DB1)	Chaque changement de statut crée une ligne dans HISTORIQUE_STATUT, testé avec Testcontainers
4.3	Requêtes de reporting (UC-DB2) : top 10 avec RANK() , évolution mensuelle avec LAG() , temps moyen de résolution, charge par technicien	Résultats corrects sur le jeu de données, projetés en DTO
4.4	Analyse et indexation des requêtes critiques (UC-DB3)	Plans d'exécution et temps avant/après consignés dans docs/PERFORMANCE.md
4.5	Package PKG_MAINTENANCE : archivage par lots (UC-DB4)	Les interventions anciennes sont déplacées vers l'archive, erreurs journalisées
4.6	Appel de la procédure d'archivage depuis Spring	Endpoint Admin qui lance l'archivage et retourne le nombre de lignes traitées
4.7	Détection et correction du problème N+1 (UC-DB5)	La liste de 20 interventions passe de 41 requêtes à 1, documenté
4.8	Scénarios de charge Gatling sur les endpoints critiques	Rapports Gatling avant/après indexation ajoutés à PERFORMANCE.md (temps de réponse, percentiles)

Sprint 5 — Tableau de bord, export & déploiement (1 semaine)

#	Tâche	Critère d'acceptation
5.1	Endpoint du tableau de bord alimenté par les requêtes de reporting	GET /api/v1/dashboard retourne tous les indicateurs en un appel
5.2	Page Angular tableau de bord avec graphiques	Interventions ouvertes par priorité, top équipements, évolution mensuelle, charge des techniciens

#	Tâche	Critère d'acceptation
5.3	Export CSV des interventions sur une période	Le fichier se télécharge depuis l'interface Admin
5.4	Dockerfiles de production (API et front), images arm64	Build multi-stage, images légères, utilisateur non-root
5.5	Docker Compose de production avec Caddy et healthchecks	<code>docker compose up</code> lance Oracle, l'API, le front et Caddy dans le bon ordre
5.6	Provisionner la VM Ampere A1 sur Oracle Cloud et le nom de domaine	VM accessible en SSH, Docker installé, domaine pointant sur l'IP publique
5.7	Pipeline CI/CD complet : tests, analyse SonarCloud, analyse de sécurité (OWASP Dependency-Check, Trivy), images sur GHCR, déploiement SSH	Un merge sur main met à jour la production automatiquement ; une vulnérabilité critique bloque le déploiement
5.8	HTTPS et sauvegarde de la base	Site accessible en HTTPS, export Data Pump planifié
5.9	Tests Angular des composants clés	Formulaires, badges et boutons d'action testés
5.10	README final : présentation, captures, schémas, lien de démo, lancement	Un recruteur teste la démo en ligne en 2 minutes, un développeur la lance en une commande

Sprint 6 — Tests end-to-end (optionnel, 1 semaine)

#	Tâche	Critère d'acceptation
6.1	Mettre en place Robot Framework avec la bibliothèque de pilotage du navigateur	Un premier test ouvre l'application et se connecte
6.2	Écrire les parcours critiques : déclaration, planification, démarrage, clôture	Le parcours complet passe pour chaque rôle
6.3	Intégrer les tests E2E à la CI	Les tests tournent sur l'environnement Docker Compose à chaque merge sur main

Planning récapitulatif

Sprint	Durée	Thème
Sprint 0	1 semaine	Modélisation Merise, contrat OpenAPI, setup, Docker, CI
Sprint 1	1 semaine	Authentification, JWT, rôles
Sprint 2	1 semaine	Référentiel équipements
Sprint 3	1 semaine	Interventions, cycle de vie en TDD, règles métier
Sprint 4	1 semaine	Oracle avancé : reporting, index, PL/SQL, performance, Gatling
Sprint 5	1 semaine	Tableau de bord, export, sécurité, déploiement sur Oracle Cloud
Sprint 6 (optionnel)	1 semaine	Tests end-to-end avec Robot Framework
Total	~6 à 7 semaines	

Notes

- Ce document sert de cahier des charges fonctionnel et de backlog produit.

- Chaque sprint produit un incrément livrable et testable.
- Les critères d'acceptation servent de base à la validation de chaque sprint.
- Hors périmètre : stock de pièces détachées, maintenance préventive planifiée, application mobile, multi-sites.
- Points à trancher avant le Sprint 0 : Flyway ou Liquibase selon la compatibilité avec Oracle Database Free.